HRSG 계산기·체크리스트 키트 — 팀 1

팀 1 페이지의 성능·경제성 계산기 와 데이터 요청 체크리스트 에 붙일 로직과 임시 데이터입니다.

두 팀이 같은 코어를 씁니다. `calc.js`, `checklist.js`, `data/`, 테스트, FORMULAS·CHECKLIST 문서는 팀 2 키트와 똑같습니다. 팀마다 다른 것은 이 README 의 연결표 뿐입니다. 모든 수치는 DEMO 값입니다.

구성

파일	내용
`calc.js`, `checklist.js`	계산기·체크리스트 함수 — JavaScript 판 (외부 라이브러리 없음)
`calc.py`, `checklist.py`	같은 함수 — Python 판 (표준 라이브러리만)
`data/calculator.json`	GT 등급 3개, 예시 운전 상태 1개, 경제성, 세정 공법 4개, 핀치 설계값, 계산 상수
`data/checklist.json`	카테고리 5개, 항목 49개 (필수 16)
`FORMULAS.md`, `CHECKLIST.md`	계산식 / 체크리스트 항목·상태·함수
`calc.test.mjs`, `checklist.test.mjs` / `test_calc.py`, `test_checklist.py`	기대값 테스트 — JavaScript 15개 (Node 18 이상) / Python 16개 (Python 3.8 이상, 추가 설치 없음)
`example.html`	어댑터 → 검사 → 계산 → 카드 의 가장 작은 예시 (JavaScript)
`README.pdf`, `CLAUDE.md`	이 문서의 PDF 판 / Claude Code 작업 규칙

팀 1 화면 연결표

함수 이름 (JavaScript ↔ Python)

두 판은 같은 식·같은 결과입니다(무작위 입력 200개로 비교해 차이 0). 결과 모양도 같아서, 이 문서의 `r.kpi.power_loss_mw` 는 Python 에서 `r['kpi']['power_loss_mw']` 입니다.

JavaScript (`calc.js`, `checklist.js`)	Python (`calc.py`, `checklist.py`)
`fetch('data/calculator.json')` 등으로 읽기	`load_calculator_data()`, `load_checklist_data()`
`defaultInputs` · `designInputs` · `validateInputs` · `calculate`	`default_inputs` · `design_inputs` · `validate_inputs` · `calculate`
`compareMethods(inp, D, 'payback')` · `lossCurve(inp, D, { param })` · `efficiencyTrend`	`compare_methods(inp, D, 'payback')` · `loss_curve(inp, D, param=..)` · `efficiency_trend`
`keyIndicators` · `pinchSection` · `formatKrw`	`key_indicators` · `pinch_section` · `format_krw`
`validateProgress` · `kpis` · `byCategory` · `summary` · `distribution` · `matrix`	`validate_progress` · `kpis` · `by_category` · `summary` · `distribution` · `matrix`
`filterItems(C, prog, { requiredOnly })` · `priorityLabel` · `detailLine` · `measureCell`	`filter_items(C, prog, required_only=True)` · `priority_label` · `detail_line` · `measure_cell`
`emailDraft(C, prog, { ids })` · `exportRows` · `importReply` · `resetProgress`	`email_draft(C, prog, ids=..)` · `export_rows` · `import_reply` · `reset_progress`

계산기 입력 (내 화면 → 키트 입력 inp)

팀 1 화면	키트 입력	비고
가스터빈 등급 (F-Class)	`equipment.gt_class`	`"F"` / `"H"` / `"E"`
정격 용량 (MW)	`equipment.rated_gt_mw`	ST 정격은 등급 값 사용
운전 부하율 (%)	`equipment.load_pct`	
현재 배압 (mmH2O)	`equipment.gt_bp_mmh2o`	단위는 키 이름으로 — 직접 변환하지 않음
기준 배압 (mmH2O)	`equipment.base_gt_bp_mmh2o`	
스택 온도 / 기준 스택 온도 (°C)	`equipment.stack_c` / `equipment.base_stack_c`	
HRSG 가스측 차압 (kPa)	`equipment.hrsg_dp_kpa`	이제 손실 계산에 쓰임
외기 온도 (°C)	`equipment.ambient_c`	
월간 가동시간 (h)	`economics.monthly_hours`	연간은 없으면 ×12
SMP (원/kWh)	`economics.price_krw_per_kwh`	
목표 회수 출력 (MW)	`economics.target_recovery_mw`	공법 회수 출력이 비었을 때 사용
공법별 비용·정지일수·간접 비용·회수 출력	`methods[i].cost_krw` · `outage_days` · `indirect_krw` · `recovery_mw`	기본 데이터는 공법 4개 + 회수율
B. 열역학	`sections.HP.drum_barg` · `evap_outlet_gas_c` · `econ_outlet_water_c` · `pinch_design_c` · `approach_design_c`	

계산기 결과 (키트 결과 → 내 화면)

팀 1 화면	키트 결과 (`r = calculate(inp, D)`)
현재 배압 · 초과 배압	`r.kpi.current_bp.mmh2o` · `r.kpi.excess_bp.mmh2o`
발전 출력 손실 · 월간 발전 손실액	`r.kpi.power_loss_mw` · `r.kpi.monthly_loss_krw`
세정 예상 회수 출력 · 세정 투자 비용(공법 이름)	`r.kpi.expected_recovery_mw` · `r.kpi.investment_krw` (`r.kpi.method_name`)
월간 추가 편익 · 순편익(첫 1개월)	`r.kpi.monthly_benefit_krw` · `r.kpi.first_month_net_krw`
투자 회수기간 · 손익분기 일수	`r.kpi.payback_months` · `r.kpi.breakeven_days` (null → 「산출 불가」)
경고 줄	`r.warnings`
배압 상태 게이지 · 세정 필요도 지수 (배압 기여 · 온도 기여)	`r.gauge` · `r.index.value` (`pressure_part` · `temp_part`)
배압-손실 곡선	`lossCurve(inp, D, { param: 'gt_bp' })` — x 축 `excess.mmh2o`
세정 공법별 비교 (회수기간 / 총투자 / 첫 달 순편익 탭)	`compareMethods(inp, D, 'payback' | 'total' | 'net')`
기본값 초기화	`defaultInputs(D)`

금액은 `formatKrw()` 로 「1.86억 원」「3204만 원」처럼 표시합니다.

체크리스트

팀 1 화면	키트
KPI 카드 4개	`kpis()` — `received` · `reviewed` · `verified` · `required`
카테고리별 수령률 · 수급 상태 분포 · 범주 × 수급 상태	`byCategory()` · `distribution()` · `matrix()`
검색 · 카테고리 · 상태 · 필수만	`filterItems({ query, category, supply, requiredOnly })`
필수·우선 · 측정값	`priorityLabel(C, item)` · `measureCell()`
요청 메일 초안 (n)	`emailDraft(C, prog, { ids: selectedIds })` (표에서 체크한 ID)
Excel 내보내기 · 표준 양식 · 회신 가져오기	`exportRows()` · `exportRows(.., { template: true })` · `importReply()`

지금 화면과 달라지는 점

- 항목 40개 → 49개: A-06, B-10~B-13, C-09, D. 경제성(D-01~D-03) 추가. 「E. 확장 (기존 HRSG 앱)」 이름은 「E. 확장」.
- 수급 상태에 해당 없음(na) 추가 — 비율의 분모에서 빠짐.
- 출력 손실에 스택 온도(ST 손실)와 HRSG 차압도 들어가고, 세정 공법 기본 데이터는 4개(기계식·고압수·드라이아이스·화학, 회수율 포함)입니다.

내 프로젝트에 넣기 (Claude Code, 한 번에)

1. 키트 넣기: 받은 `hrsg_calc_kit_team1.zip` 을 내 웹 프로젝트 폴더 안에 복사합니다. 이 PDF 를 보려고 이미 압축을 풀었다면 `hrsg_calc_kit_team1` 폴더를 통째로 넣어도 됩니다.
2. 저장해 두기: git 을 쓰고 있다면 지금 상태를 commit 해 둡니다. 결과가 마음에 들지 않으면 되돌릴 수 있습니다.
3. Claude Code 열기: 그 폴더에서 Claude Code 를 엽니다(VS Code 로 폴더를 열고 Claude Code 실행, 또는 터미널에서 그 폴더로 이동해 `claude`).
4. 프롬프트 붙여 넣기: 아래 프롬프트 하나를 그대로 붙여 넣습니다. 파일 수정 허가를 물으면 허용합니다. Claude Code 가 아래 「작업 목록」을 모두 하고, 「기대값」으로 스스로 확인한 뒤 결과 표를 보여 줍니다.

프롬프트 1 · 처음 한 번 붙여 넣기 — 상자 안의 글 전체를 복사해 붙여 넣기

```
이 프로젝트에 hrsg_calc_kit_team1.zip 이 있으면 hrsg_calc_kit_team1 폴더로 풀고(이미 폴더가 있으면 그대로 써 줘),
폴더 안의 파일 구조는 바꾸지 마.
그 폴더의 README.md, FORMULAS.md, CHECKLIST.md, CLAUDE.md 를 읽고,
먼저 이 프로젝트의 언어와 프레임워크를 확인해서 알려 줘. JavaScript/TypeScript 면 calc.js · checklist.js 를,
Python(Streamlit·Flask 등)이면 calc.py · checklist.py 를 그대로 쓰고(README 「함수 이름」 표),
그 밖의 언어면 calc.js · checklist.js 를 그대로 옮긴 뒤 키트 테스트와 같은 입력에서 같은 기대값이 나오는지 확인해 줘.
그 언어의 키트 테스트가 통과하는지 확인해 줘(JavaScript: node --test hrsg_calc_kit_team1/calc.test.mjs
  hrsg_calc_kit_team1/checklist.test.mjs,
Python: python -m unittest discover -s hrsg_calc_kit_team1 -p "test_*.py").
그다음 README 의 「팀 1 화면 연결표」와 「작업 목록」대로 내 계산기와 체크리스트 화면에 키트의 데이터와 로직을
  모두 연결해 줘.
- 내 화면 상태를 키트 입력으로 바꾸는 어댑터 함수 하나를 만들고, validateInputs() 의 오류·경고가 0개가 되게 해 줘.
- 화면 구성과 디자인은 바꾸지 말고, 내 화면에 없는 요소는 새로 만들지 말고 건너뛰어 줘.
- hrsg_calc_kit_team1 폴더 안의 파일은 고치지 마.
끝나면 README 의 「기대값」 표의 값이 화면에 나오는지 하나씩 확인해서 (항목, 기대값, 내 화면 값, 일치 여부) 표로
  보여 주고,
연결하지 못했거나 건너뛴 것이 있으면 이유와 함께 알려 줘.
```

값이 다르거나 오류가 나면 이어서 붙여 넣습니다.

프롬프트 2 · 값이 다르거나 오류가 날 때만 — 상자 안의 글 전체를 복사해 붙여 넣기

```
화면 값이 README 의 기대값과 다르거나 오류가 나. 어댑터가 만든 inp 를 콘솔에 출력하고 validateInputs(inp, D) 의
errors 와 warnings 를 보여 준 다음, README 연결표와 비교해서 원인을 설명해 줘. 코어 파일은 고치지 말고 어댑터만
  고쳐 줘.
```

작업 목록

계산기 (성능·경제성 계산기)

1. 키트 불러오기: `calc.js` 또는 `calc.py` 와 `data/calculator.json` (Python 은 `load_calculator_data()`). 입력 변경·「계산 실행」 때 `calculate(inp, D)` 를 부르고, 「기본값 초기화」는 `defaultInputs(D)`.
2. KPI 카드 10개 = `r.kpi` (압력은 `.mmh2o`, 금액은 `formatKrw`, null 이면 「산출 불가」).
3. 경고 줄 = `r.warnings`. 배압 상태 게이지 = `r.gauge` (값·최대·공개 참고 구간, `mmh2o`). 세정 필요도 = `r.index` (배압 기여 `pressure_part`, 온도 기여 `temp_part`).
4. 배압-손실 곡선 = `lossCurve(inp, D, { param: 'gt_bp' })` — x 축 `excess.mmh2o`, y 축 `loss_mw`, 현재 위치 `r.kpi.excess_bp.mmh2o`.
5. 공법별 입력란 = `inp.methods` (데이터의 공법 4개 모두). 비교표 = `compareMethods(inp, D, 'payback' | 'total' | 'net')` — 탭 「회수기간」「총투자」「첫 달 순편익」. 열: 비용·정지일수·회수 출력·월간 편익·정지 손실·총투자·회수기간·첫 달 순편익. `available` 이 false 면 `message` 표시. 「회수기간이 가장 짧은 공법 기준 (가정 기준, 추천 아님)」 문구 유지.

6. B. 열역학 = `inp.sections.HP` 입력, 결과 `pinchSection('HP', inp.sections.HP, D)` (핀치·어프로치·판정·문구).

체크리스트 (데이터 요청 체크리스트)

7. 항목 = `data/checklist.json` 의 49개. 진행 상태는 `{ "A-01": { supply, verify, reviewed, value, note } }` 로 저장(브라우저 앱은 localStorage, Python 앱은 세션 상태와 JSON 파일 — 예: Streamlit `st.session_state` + `progress.json`)하고, 불러올 때 `validateProgress()` 오류가 있는 항목만 처음 상태로.
8. KPI 카드 4개 = `kpis()`, 카테고리별 수령률 = `byCategory()`, 수급 상태 분포·미수령 필수 항목 = `distribution()`, 범주 × 수급 상태 = `matrix()` (칸 클릭 → `filterItems({ category, supply })`).
9. 표: 단위 `unit`, 회색 줄 `reason`, 필수·우선 `priorityLabel(C, item)`, 측정값 `measureCell()`, 드롭다운 글자는 `C.supply_states` · `C.verify_states` 로 보여 주고 키로 저장.
검색·카테고리·상태·필수만 = `filterItems()`, 「n/49」 = 결과 수 / 전체.
10. 「요청 메일 초안 (n)」 = 체크한 ID 로 `emailDraft(C, progress, { ids })` — 복사만, 메일은 보내지 않음. 「Excel 내보내기」 = `HEADERS` + `exportRows()`, 「표준 양식」 = `exportRows(.., { template: true })`, 「회신 Excel 가져오기」 = `importReply()` (xlsx 읽기·쓰기는 JavaScript 면 SheetJS, Python 이면 openpyxl 또는 pandas).

기대값

화면	상태	기대값
KPI 카드	기본값 초기화	현재 배압 305.9 mmH2O · 초과 배압 51.0 mmH2O · 출력 손실 1.89 MW · 월간 손실 2.04억 원 · 회수 출력 1.32 MW · 투자 4.46억 원(드라이아이스 세정) · 월간 편익 1.43억 원 · 첫 달 순편익 -3.03억 원 · 회수기간 3.12개월 · 손익분기 93.6일
경고 · 게이지 · 세정 필요도	기본값	경고 없음 · 51.0 / 203.9 mmH2O, 공개 참고 76.2~101.6 · 41.1 / 100 (배압 33.3 · 온도 7.8)
배압-손실 곡선	기본값	초과 0 에서 1.89 MW → 203.9 mmH2O 에서 5.13 MW
공법 비교	기본값	회수기간 탭: 드라이아이스 3.12 → 고압수 3.30 → 기계식 3.54 → 화학 4.27개월 · 총투자 탭: 기계식 3.25억 → 고압수 4.04억 → 드라이아이스 4.46억 → 화학 7.42억 원
공법 비교	공법 하나의 비용을 지움	「산출 불가 — 입력 필요: 세정 비용」
B. 열역학	기본값	핀치 8.0 °C · 어프로치 5.0 °C · 「설계 범위 안입니다.」
체크리스트	처음	수령률 0% 0/49 · 필수 충족 0/16 · A 0/6 · B 0/13 · C 0/9 · D 0/3 · E 0/18 · 실측 미입력 49 · 미수령 필수 16개 · 「49/49」 · 필수만 16 · 「핀치」 검색 5건 · B-09 측정값 「문서」
체크리스트	A-01 을 수령 완료로	수령률 2% (1/49), 새로고침해도 유지
메일·Excel	A-02 체크	「요청 메일 초안 (1)」, 문구에 「- A-02 운전 배압 … 〔필수〕」 · 표준 양식에 A-02 측정값을 채워 가져오면 A-02 가 「검증 필요」

칼럼이 달라서 생기는 오류 막기

- `validateInputs(inp, D)`: 빠진 값·숫자가 아닌 값·모르는 GT 등급·같은 압력을 두 단위로 넣은 경우는 오류, 모르는 이름·단위가 의심되는 값은 경고.
- `calculate()` 는 오류가 있으면 계산하지 않고, 무엇이 빠졌는지 적힌 오류를 냅니다(화면에 NaN 이 나오지 않음). `<input>` 의 문자열 값은 숫자로 바꿔 계산합니다.
- `validateProgress(C, progress)`: 모르는 항목 ID, 화면 글자("수령 완료")를 그대로 저장한 상태 값을 알려 줍니다.

규칙

- 코어(`calc.js`·`calc.py`, `checklist.js`·`checklist.py`, `data/`)는 고치지 않습니다. 이름·단위 차이는 어댑터에서 바꿉니다.
- 공법은 「회수기간이 가장 짧은 공법 (가정 기준, 추천 아님)」으로 표시합니다.
- 발전소 이름·측정값·메모는 LLM 에 보내지 않습니다. 이 키트를 외부에 다시 배포하지 않습니다.
- `example.html`(JavaScript 예시)은 키트 폴더에서 `npx serve .` 또는 `python3 -m http.server` 로 엽니다.